

SEAL: Structured Encoder for Ancient Logograms

Emanuele Messina

giuseppeemanuele.messina@studenti.polito.it

Salvatore Cimmino

salvatore.cimmino@studenti.polito.it

Abstract

Most OCR softwares do not support chinese seal script characters, which hold significant cultural and historical value. In this paper, we present our contributions towards developing a custom detector capable of detecting thousands of these characters, along with a comprehensive dataset. Our model leverages the intrinsic hierarchical nature of chinese characters to reduce the latent space dimension compared to a standard classifier. This allows for a more compact model with feasible training and inference times while achieving reasonable accuracy on 5684 classes – while a non hierarchical version of our model and YOLO11x could not converge at all.

1. Introduction

Seal script [5] is an ancient style of Chinese writing that dates back over 2000 years. It represents a critical stage in the evolution of Chinese characters, serving as the foundation for modern script forms. Despite its historical origins, seal script remains widely used today (mostly for decorative purposes) in official seals [6], calligraphy, and cultural artifacts such as inscriptions, stone carvings, and historical documents. Automatic seal script recognition is valuable for historical research, digital preservation, and modern applications like seal authentication and design. However, its unique structure and stylistic variations make it challenging for conventional OCR systems, requiring specialized approaches.

Given the vast number of classical chinese characters (more than 40k), and due to our limited resources, we aimed to develop a model capable of recognizing approximately the same number of characters used in modern chinese (around 5k). This number is far beyond the typical number of classes (e.g. ImageNet-1k) used in standard object recognition. A conventional classifier would struggle with this task, as it would require an extremely large feature space to differentiate thousands of characters independently.

To address this, we leveraged the intrinsic hierarchical structure of Chinese characters, which consist of a radical and a remaining component. We designed a deep hierar-

chical classifier where a shared feature tensor is used by both a radical classifier and multiple radical-specific classifier heads. Ideally, each head is responsible for distinguishing characters within a single radical category, utilizing both the shared feature tensor and the radical classification results. This structure significantly reduces the required feature space while allowing for a combinatorial increase in the number of recognizable classes.

Compared to a standard (non hierarchical) version of our model (where the upper radical classifier was disabled), which collapses under this task, our hierarchical approach maintains feasible training and inference times while achieving a reasonable classification and localization performance for a minor increase in size (due to the added radical classifier).

To recognize seal script characters, we first needed a dataset, as - to our knowledge - no existing model or dataset was available, at least publicly. Thus we had to invent our own: SEAL 5684.

2. Data

We based our dataset on the Kangxi Dictionary [4], one of the most comprehensive Chinese character dictionaries. It contains over 40000 characters, many of which are obscure or no longer in common use, so we first isolated characters that are included in the Unicode CJK set, reducing the dataset to 16000+ characters. Next, we identified 18 distinct fonts capable of rendering these characters in a seal script style. We then removed characters that could not be rendered by any of these fonts, leaving 13000+ characters, and further filtered out those represented by fewer than half of the available fonts. This ensured sufficient style variations for each character, ultimately resulting in 5684 characters for our dataset. To ensure a diverse dataset, we incorporated offline data augmentation, avoiding slowdowns during training, since the dataset had to be large anyway, due to the high number of classes. The use of different fonts was actually a type of augmentation itself. We generated synthetic images using 12 different backgrounds, with characters placed in random sized vertical, square, and single arrangements, mimicking real-world usage in documents and seals. Each character underwent a random perspective

deformation, and 2D rotation for added variation. Images were sized at 256×256 pixels, containing 1 to 7 characters per image. To estimate the number of images required, we used the coupon collector’s problem [3], determining the necessary number of random draws to ensure all characters appeared at least once. Given an average of 3.5 characters per image, we rounded up our calculations to 35000 generated images. Since both the dataset and model were experimental, we prioritized convergence over extensive validation. We performed a simple 90% – 10% split, generating 4,000 test images. For labeling, we created a custom annotation format tailored to our hierarchical model. Additionally, we generated a YOLO-compatible label set, which does not incorporate hierarchical classification. We computed the mean and standard deviation of the dataset to normalize inputs during training. Lastly, we analyzed the distribution of characters and radicals, finding a strong bias toward common radicals (1). To mitigate this imbalance, we calculated class weights for both radicals and individual characters, which were later used in a weighted cross-entropy loss function during training, according to the following formula [2]:

$$w_c = \frac{\sum_{i \in C, s \in S} n_i^s}{|C|n_c}$$

where w_c is the weight assigned to class c , C is the classes set and $|C|$ is the number of classes, n_c is the absolute frequency of class c over the whole dataset, S is the dataset and n_i^s is the frequency of class i in sample s (meaning the numerator is the total number of instances).

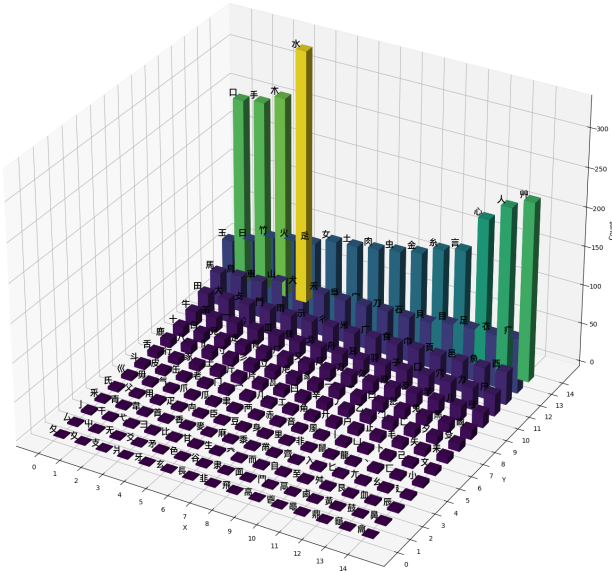


Figure 1. Radicals distribution over the train dataset. The dataset contains all 214 of them.

3. Methods

3.1. Model Design

3.1.1 Base model

Given the high number of classes and the detailed features needed to distinguish one character from another, we hypothesized that single-shot detection models (e.g., YOLO) would not be ideal. Unlike standard object detection, which deals with diverse objects in color, shape, and size, our task required fine-grained recognition within a large class space. A two stage recognition has a separate classification stage that operates on (ideally) the feature map crop that contains the entire object of interest, while a single shot method could crop important parts of a character due to the fixed grid structure, and by using a coarser grid it risks getting too many character parts in the same cell, making the character learning process difficult. While we can’t prove this conjecture in general, as we will see in the results section our two-stage model did actually perform better than YOLO, but we believe the main reason is our radical enhanced classification advantage more than the detection method used. Moreover, to build our hierarchical model, we needed a flexible framework that allowed modifications without rewriting everything from scratch (as we should have done if we opted for a single stage method). We chose Faster R-CNN with a ResNet-50-FPN backbone as our base model. We kept the backbone intact and developed a custom head after the multi-scale ROI Align layer.

3.1.2 Custom head

The original implementation of Faster R-CNN assigns a box regressor per class, which is impractical for thousands of classes, but more importantly unnecessary for our task. Instead, we used a single box regressor and a binary classifier (character vs background). Their task is to understand the idea of a character (how it generally looks like) and anything else that is not a character, without knowing which specific character it is. The reasons why we didn’t use a single box regressor but a multiclass classifier are multiple. First, we noticed there is a sort of limit in the RPN code before it breaks, depending on the number of classes. Second, the binary classifier works together with the box regressor, so it makes sense to keep the task simple for them, and that is also why we’ll see that the localization accuracy is near perfect and independent from the character classification accuracy (compared to the custom classifier, they have much more data for their simple task).

The feature tensor from the multiscale ROI align layer is shared between two branches: one goes to the box regressor + binary classifier combo (linear layers), the other goes to our custom classifier (convolutional).

3.1.3 Hierarchical classifier

The feature tensor is shared again between two branches: one goes to the radical funnel (a convolutional feature extractor), the other goes to the character funnel (another convolutional feature extractor). The radical classifier is then just a couple of linear layers after the radical funnel, and tries to classify the radical of a character. The features extracted by the radical funnel (just before the actual radical classifier layers) are then concatenated with those extracted by the character funnel. The new fat tensor, that now contains both generic character features and specific radical information, is processed by a fusion layer (made of 1x1 convolutions) before being shared among all the radical-specific subclassifier heads. The idea is that each subclassifier head handles the classification of characters (the subclasses) belonging to one radical (the superclass). In practice, since the radical distribution is highly imbalanced (1), we automatically grouped less frequent radicals together so that each subclassifier has to classify more or less the same number of characters. This allows also for a more generalized development. A subclassifier is just a single linear layer that classifies its specific group of characters directly from the fat tensor. When we refer to the non hierarchical version of our model, we mean that the subclassifiers receive just the general character features, not the fat tensor, because the radical features do not get concatenated.

3.2. Loss

We preserved Faster R-CNN’s original loss for the binary classifier (character vs. background) + box regressor, as they contain separate information from our classification stage. We separately computed radical classification loss and character classification loss (by concatenating the logits of all subclassifiers), using weighted Cross Entropy losses. These losses were weighted differently to balance the priority of each task:

$$L = L_f + \lambda_r L_r + \lambda_c L_c$$

where L is the final loss we optimize, L_f is Faster RCNN’s original loss computed on the box regressor + binary classifier outputs, L_r is the radical loss and L_c the character loss.

3.3. Training adjustments

Since the hierarchical classifier has to learn a much more complex task than the box regressor, we wanted to make sure it receives the correct ROI features for each character during training, so we take the FPN feature maps and perform the ROI align on the ground truth boxes, then feed the ROIs to the classifier. During inference, we use the RPN generated features as usual.

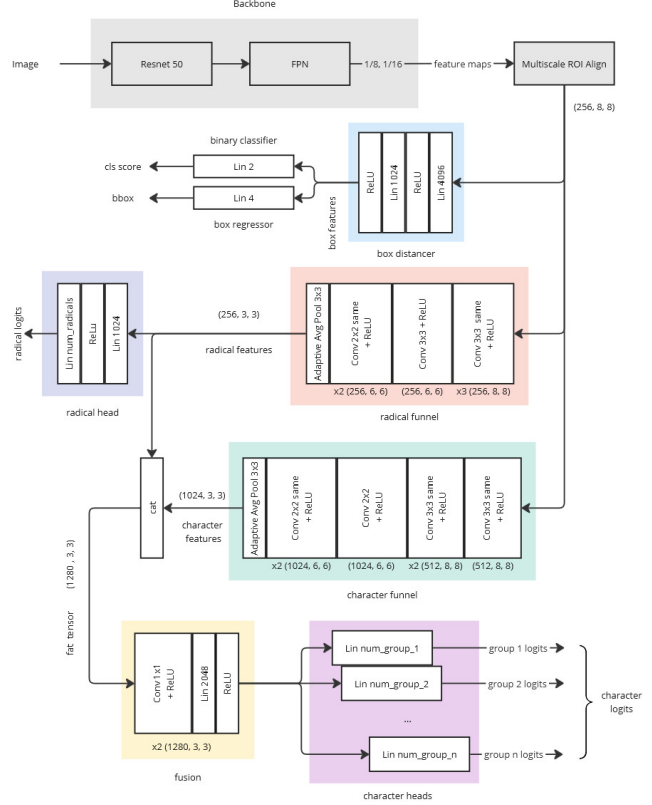


Figure 2. SEAL (our model) architecture.

4. Experiments

4.1. Early design and challenges

Initially, our model’s classifier was fully linear after ROI Align, making it bulky, slow, and inefficient. However, this simplified architecture allowed us to verify our hierarchical classification concept. During early experiments, we observed that the custom classifier gradients conflicted with the box regressor ones. Since both components shared the same feature tensor but aimed to learn different tasks, their optimization paths interfered, leading to unstable regression loss.

To solve this, we introduced additional distancing layers between the ROI features and each of the following stages. These layers acted as a low-pass filter for features, allowing the ROI features to remain general at first before progressively specializing. These layers evolved into the funnel structures seen in our final architecture (2).

4.2. Preliminary experiments

Before training on the full dataset, we tested reduced versions with 725 and 2061 classes, pruning the characters database based on a classical Chinese corpus frequency list, since we initially feared the model would not converge at all. However, it did, even without the use of hierarchical

classification, suggesting that the latent space had enough capacity for that class count. Encouraged by this result, we moved to the full dataset (almost three times larger than the last used provisory one), where we could appreciate the advantage of the hierarchical classification, which became evident: the hierarchical model converged, while the non-hierarchical version completely collapsed when trained from scratch. This validated our hypothesis that hierarchical classification was a valid option for handling thousands of characters effectively.

4.3. Optimizer selection

Initially, we experimented with Adam, hoping its adaptive learning rates and momentum tracking would help the different terminal parts of the model learn their respective tasks better. However, training was unstable, with losses skyrocketing, even with gradient clipping (that we kept on with max norm of 10 from that moment on).

Switching to SGD with momentum 0.9 and weight decay 0.0001 proved significantly more stable and eventually provided good results for the final model. After tuning learning rates between 0.01 and 0.001, we found 0.001 to be optimal.

To further stabilize training, we applied a StepLR scheduler after every epoch, testing different decay factors from 0.1 up. A gamma of 0.8 provided the steadiest loss descent.

4.4. Training strategy

The batch size was 2 for every test, as it is normal for a two stage model, considering that for every image there are on average 3 characters, and the number of proposals from the RPN is several hundreds per image. The model was trained for 6 epochs. It is worth noting that the linear model took 3 epochs before reaching its limit, with worse results (roughly 10% less mAP than our final model, and being 3 times heavier).

4.5. Loss coefficients

Given the previous issues with regression vs classification conflicts, we gave the highest priority to the Faster R-CNN's built in loss (box regression + binary classifier) by assigning a coefficient of 1. Second in priority we put the radical classification, since it predicts the high level classes on which the subclassifiers depend, with a coefficient of 0.99 (λ_r). Since the correctness of every subclassifier head is highly dependent on the correctness of the radical classifier, we gave it a coefficient of 0.95 (λ_c) to give more priority to the radical classification.

4.6. Architecture details

We use 8x8 ROI maps (with 256 channels returned by ResNet), aligned from just the 8x and 16x downsampled feature maps (P2 and P3 layers) of FPN (for 256x256 images they correspond to 32x32 and 16x16 feature map

sizes). We chose these feature maps based on the known size of the characters (from 40 to 80px) in the images. P4 and P5 (32x and 64x) are more indicated for larger objects and this is not our case. For the anchors we use sizes 50 and 70 px with 1:1 (square) and 2:1 (vertical) aspect ratios, resembling the shapes of the characters.

For the final convolutional model, all the details are reported in the architecture diagram (2).

The idea for the funnels is to try to extract more task specific features from the shared ROI features to help the later stages and avoid backpropagation conflicts, using same convolutions to avoid reducing the map size and expanding the channels in the character funnel. We don't expand the channels in the radical funnel since we deemed them sufficient for the number of radicals present. The final number of channels for the character features is 1024 with a final map of 3x3, which compared to the number of classes, would seem pretty low if hierarchical classification was not used, even when concatenated with the radical features, reaching only 1280 channels. Before going into the subclassification heads, the fat tensor goes through a couple of 1x1 convolutions to provide a touch of spatial information exchange between the character features and the radical features. The model has 145M parameters and weighs 1GB.

Hyperparameter	Value
Optimizer	SGD
Momentum	0.9
Weight Decay	0.0001
Learning Rate	0.001
Learning Rate Scheduler	StepLR
Gamma	0.8
Gradient Clipping Max Norm	10
Batch Size	2
Epochs	6
λ_r	0.99
λ_c	0.95

Table 1. Summary of training hyperparameters

Property	Value
ROI Map Size	8x8
ROI sampling ratio	2
Feature Maps	P2 (8x), P3 (16x)
Anchor Sizes	50, 70 px
Anchor Aspect Ratios	1:1, 2:1
Model Parameters	145M
Model Size	1GB

Table 2. Summary of model properties

5. Results

5.1. Qualitative results

Qualitative results are best discussed directly in the images starting from 7, where we provided results over the test dataset and real world images (thus extracted from a different distribution than the one the model was trained on). We provide confusion matrices for both the hierarchical and non hierarchical version of our model, to show that while the hierarchical version successfully understands the task, the non hierarchical version does not.

5.1.1 YOLO11x test

We trained YOLO11x (the biggest and most recent model currently available at the time of this study) for the same number of epochs. Training of YOLO has been made very simple nowadays by the developers, with the model being highly advanced and capable of setting its hyperparameters on the fly by doing a preliminary automated analysis on the dataset. There is no hard limit on the number of classes YOLO can be trained on, but the higher the classes the lower the performance will likely be [1]. That is indeed the case for our test, where the confusion matrix is not even shown because it is just a white, empty square. Of course our test it is not exhaustive, as we didn't try to manually set every hyperparameter and did not allow the model to train longer than ours, but we believe this test proves a point nonetheless: probably the architecture of YOLO and similar single shot detectors is not suited for this specific task.

5.2. Evaluation metrics

We provide the Mean Average Precision @ 0.75 IOU and the Mean Average Recall over the test split as evaluation metrics for our model. Since the localization accuracy is qualitatively really high, we fixed the IOU threshold as 0.75 when calculating the Mean Average Precision for each class. Essentially, in our case mAP is a strong indicator of classification performance. We followed the standard detection conventions when calculating mAP: a non matched predicted bounding box counted as a false positive; a non matched target box counted as false negative; a matched box with the wrong label is still a false positive, and a matched box with the correct label is a true positive. The Mean Average Recall is more meaningful as a metric for localization since it is worsened by the presence of false negatives, that in our case are rare due to the well behaving localization, and in fact we can see it is higher than the mAP, which is worsened by the character and radical classification mismatches, that raise the false positives count.

Metric	Value
mAP@0.75	0.57
mAR	0.65

Table 3. Summary of evaluation metrics over the test dataset.

6. Future work

We are satisfied with the results given the experimental nature of this project. However, there is much to be improved. First of all, it would be appropriate to conduct an ablation study, for example in the case of the fusion stage, to verify its actual usefulness. Other tests to see if the accuracy can be improved are certainly in order, by adding or modifying the existing layers. Extensive experiments with other architectures would be interesting to confirm or dismiss the validity of hierarchical classification. This particular version of our model could be fine tuned with images containing characters of different colors, deteriorations, and different backgrounds, to see if it can generalize to be used in a real world scenario. Finally, we wonder if the knowledge of this model could be distilled into a smaller model with a straightforward classifier while maintaining accuracy.

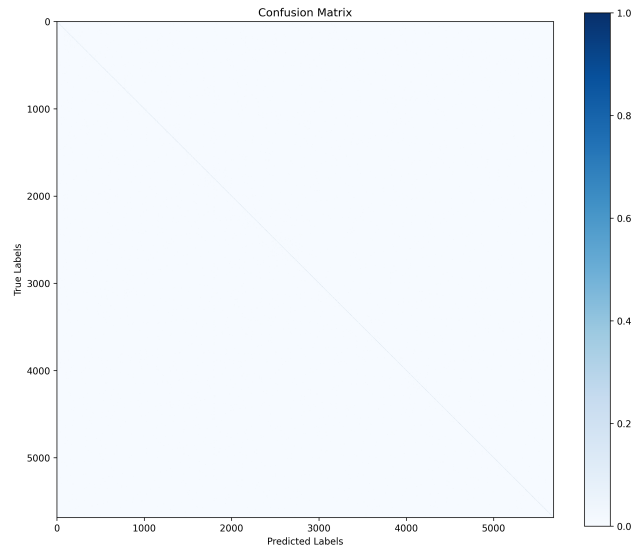


Figure 3. Confusion matrix (hierarchical) for characters, log scale, normalized. The expected diagonal is visible, indicating task accomplishment.

References

- [1] GitHub. Ultralytics Issue 1130.
- [2] Gary King and Langche Zeng. Logistic regression in rare events data. *Political Analysis*, 9, 09 2002.
- [3] Wikipedia. Coupon collector's problem.
- [4] Wikipedia. Kangxi Dictionary.
- [5] Wikipedia. Seal script.
- [6] Wikipedia. Seals in the Sinosphere.

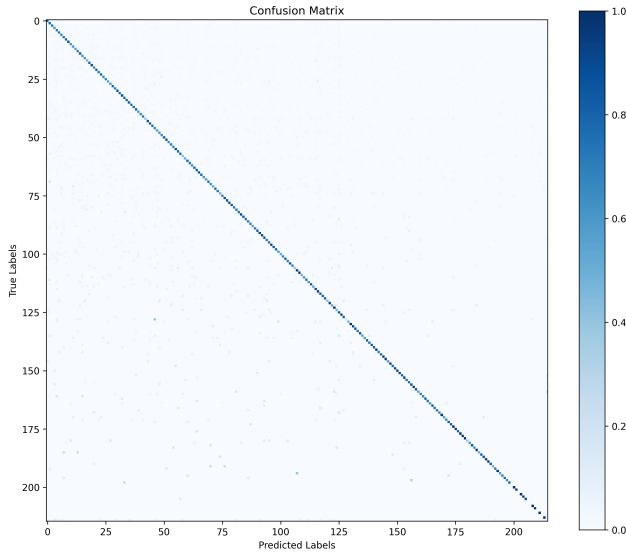


Figure 4. Confusion matrix (hierarchical) for radicals, log scale, normalized. The expected diagonal is visible, indicating that the radical classifier is working well.

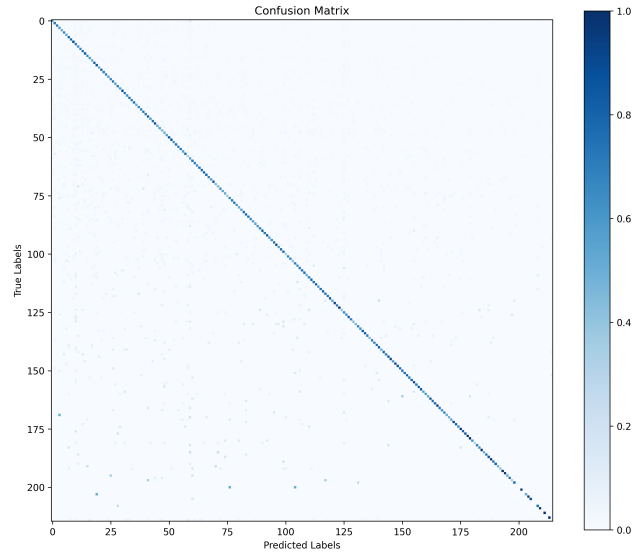


Figure 6. Confusion matrix (non-hierarchical) for radicals, log scale, normalized. The expected diagonal is visible, indicating that the radical classifier is working well and it does not depend on the subclassifiers.

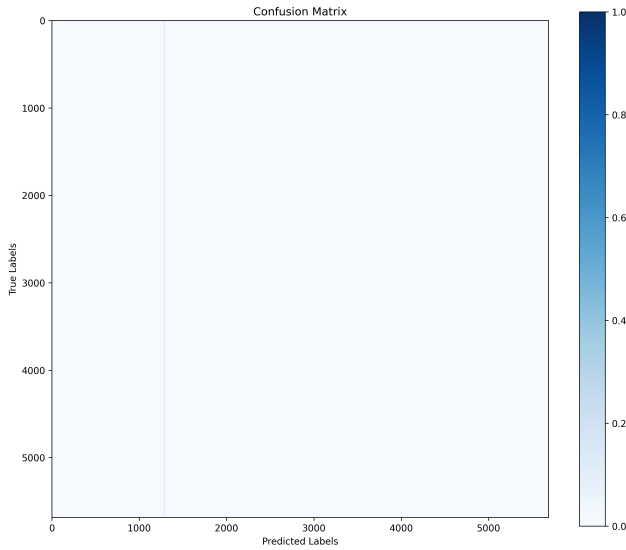


Figure 5. Confusion matrix (non-hierarchical) for characters, log scale, normalized. Not only there is no diagonal, but instead there is a complete collapse of the output distribution, indicating failure.

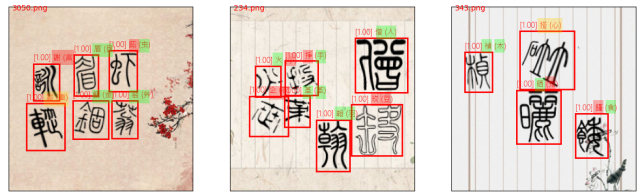


Figure 7. Some predictions from our model on the test set. Over each bounding box there are reported the confidence score the predicted character and the predicted radical. A red highlight indicates a mismatch in the prediction. Green indicates a correct prediction. Yellow indicates a prediction that is technically correct but there was a dataset error (a few fonts rendered some wrong characters) so it is wrong in the real world.

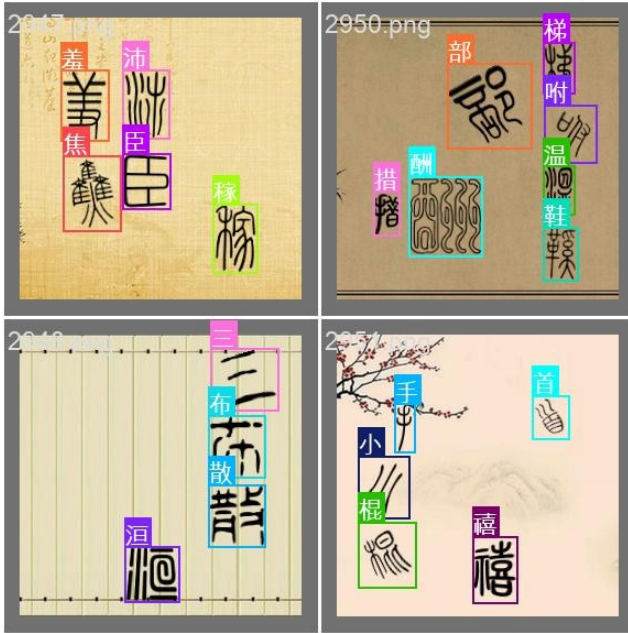


Figure 8. Some samples from YOLO11x training, that it had to predict, with the corresponding ground truth classes highlighted.

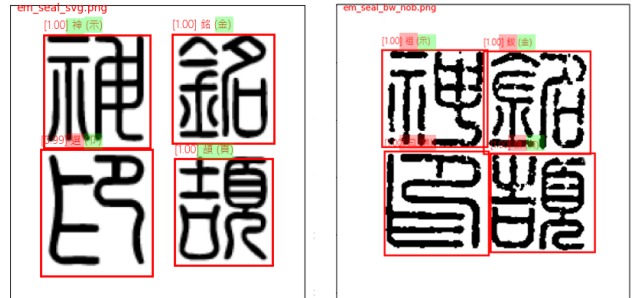


Figure 10. Image from a different distribution than what the model was trained on. The same highlighting convention as in image 7 applies. The left image is artificial, we can see that the model is very accurate. The right image is a photograph of a print of the same characters in the left image, with a different style and deterioration. We can see that the character classification struggle with increasing deterioration while the radical classification is immune, suggesting the radical classifier generalized successfully. In the character classifier defense, we can say that first, its task was much bigger, and second, the predicted characters are visually very similar to the ground truths, so it is not a disaster at least.



Figure 9. YOLO11x predictions for image 8. Only one character is predicted, with low probability.

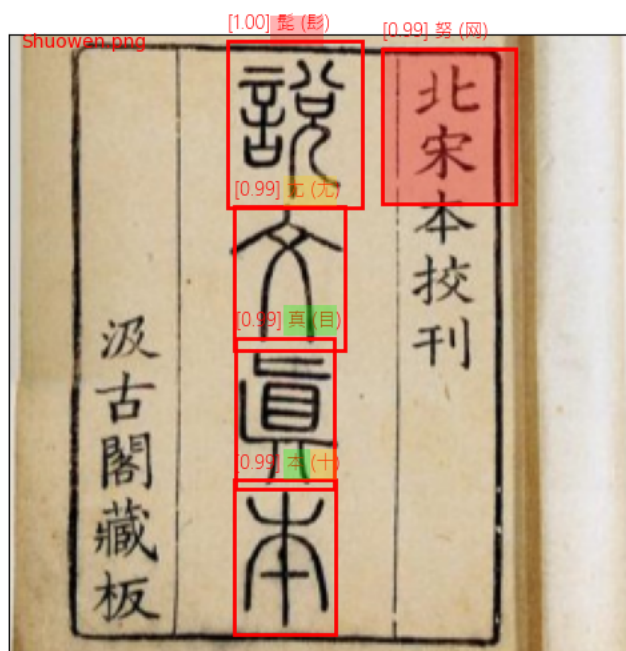


Figure 11. A real life image from the Shuowen Jiezi cover (which counts as yet a different distribution than what the model was trained on, since it never saw real life images). In this case an entire box was highlighted in red to indicate a false positive character presence. It is nice to see that even though the model was not exposed to modern character forms, it correctly localizes just the seal script characters in the center and not the modern forms on the side, except for the upper right red box. Among the four seal characters detected, the two green ones are correct (the yellow predicted radical is wrong, but in the model's defense, it is a visually reasonable mismatch), the red one is wrong, and the yellow one, although it is technically wrong, it is visually similar to the correct one, so we are still satisfied with the result for it being a first time ever.